

Qdrant Cheat Sheet

Current 1.x line (latest 1.19.1). Quick reference only — verify against the official docs before production use.

COLLECTION & POINT MODEL

- **Collection:** fixed vector size(s) + distance metric per named vector
- **Point:** id (uint64/UUID) + vector + payload (JSON)
- **Named vectors:** multiple vectors per point, one config each
- **Multivectors:** a point vector as a matrix (e.g. ColBERT late interaction)
- **Sparse vectors:** named sparse config, index/value pairs

DISTANCE METRICS

Metric	Use case
Cosine	direction only; most embeddings
Dot	pre-normalized vectors, max throughput
Euclid	raw L2 magnitude matters
Manhattan	L1, sparse/robust-to-outliers features

Set once per (named) vector at collection-creation time; cannot be changed later.

HNSW PARAMETERS

Param	Meaning
m	edges per node (build-time recall/size)
ef_construct	candidate list size while building
ef / hnsw_ef	candidate list size at search time

Higher m/ef_construct/ef → better recall, more memory/latency. exact: true forces brute-force search.
Quantization: scalar (int8), product (PQ), binary — trade recall for RAM/speed.

FILTER-CLAUSE SYNTAX

```
{
  "must": [{ "key": "city", "match": { "value": "NYC" } }],
  "should": [{ "key": "tag", "match": { "any":
    ["a", "b"] } }],
  "must_not": [{ "key": "price", "range": { "gt": 100 } }]
```

Conditions: match, range, geo_radius, geo_bounding_box, is_empty/is_null.
Payload indexes (keyword/integer/geo/text) make filtered HNSW search fast.

REST ONE-LINERS

```
PUT /collections/c
  {"vectors":{"size":768,"distance":"Cosine"}}
PUT /collections/c/points
  {"points":[{"id":1,"vector":[...],"payload":{}}]}
POST /collections/c/points/search
  {"vector":[...],"limit":10,"filter":{...}}
POST /collections/c/points/query
  {"query":[...],"filter":{...}}
```

GRPC ONE-LINERS

```
Collections.Create(name="c",
  vectors_config=VectorParams(size=768,
    distance=Cosine))
Points.Upsert(collection_name="c", points=[...])
Points.Search(collection_name="c",
  vector=[...], limit=10, filter=...)
Points.Query(collection_name="c",
  query=[...], filter=...)
```

PYTHON (QDRANT-CLIENT)

```
client.create_collection("c",
  vectors_config=VectorParams(size=768,
    distance=Distance.COSINE))
client.upsert("c", points=[PointStruct(id=1, vector=[...],
  payload={})])
client.query_points("c", query=[...],
  query_filter=Filter(...), limit=10)
```

SCROLL API (PAGINATION)

```
client.scroll(
  collection_name="c",
  scroll_filter=Filter(...),
  limit=100,
  offset=None,           # or next_page_offset
  with_payload=True,
  with_vectors=False,
) -> (points, next_page_offset)
```

Loop until next_page_offset is None to walk an entire collection.

CRUD & BATCH OPS

- upsert / retrieve / delete — single or batched points
- Batch upsert/delete trade per-call overhead for throughput
- Point IDs: uint64 or UUID, chosen at insert time

HYBRID & SPARSE SEARCH

```
client.query_points("c",
  prefetch=[Prefetch(query=sparse_vec, using="text-sparse",
    limit=50),
    Prefetch(query=dense_vec, using="dense",
    limit=50)]),
  query=FusionQuery(fusion=Fusion.RRF), limit=10)
```

Query API prefetch fuses dense + sparse (SPLADE/BM25-like) via Reciprocal Rank Fusion, or re-ranks with a cross-encoder.

CLUSTERING & STORAGE

shard_number / replication_factor at collection creation; Raft consensus for cluster metadata.
Storage: in-memory (fast) vs. on-disk/mmap (large collections); snapshots for backup/restore.

SECURITY & ADMINISTRATION

Static API keys (read-write / read-only) via api-key header; TLS for REST/gRPC.
Qdrant Cloud adds RBAC. /metrics (Prometheus format) and /telemetry for monitoring.

RAG INTEGRATION

Chunk + embed corpus → store as points (payload = source metadata) → retrieve top-k → optional re-rank → feed LLM context.
Wiring: LangChain QdrantVectorStore, LlamaIndex QdrantVectorStore.